

Oppgave 2.1 (brukte KI og W3Schools som hjelpemidler. generert lenke fantes og fungerte ikke, så her

hva er forskjellen mellom abstract og interface?



Programmering 2 Chat-Assistent - Egendefinert Gem

Hei! Det er et veldig sentralt og godt spørsmål i Java og objektorientert programmering. Begge deler brukes for å oppnå **abstraksjon**, men de har ulike bruksområder og regler.

Her er en oversikt over de viktigste forskjellene:

1. Tilstand (Instansvariabler)

- **Abstrakte klasser:** Kan ha vanlige variabler (fields) som holder på en tilstand (f.eks. `protected int speed;`).
- **Interface:** Kan som hovedregel ikke ha instansvariabler. Alle variabler du definerer i et interface er automatisk `public static final` (altså konstanter).

2. Implementering av metoder

- **Abstrakte klasser:** Kan ha både abstrakte metoder (uten kropp) og konkrete metoder (med kode). Dette er nyttig hvis du vil dele felles funksjonalitet mellom subklasser.
- **Interface:** Består primært av metodesignaturer. Siden Java 8 kan man også ha `default`- og `static`-metoder med implementering, men hovedformålet er å definere en kontrakt for hva en klasse kan gjøre, ikke *hvordan*.

3. Arv (Multiple Inheritance)

- **Abstrakte klasser:** En klasse kan bare arve (extend) én abstrakt klasse. Dette følger Javas regel om enkel arv.
- **Interface:** En klasse kan implementere mange interfaces samtidig. Dette gir stor fleksibilitet.

Sammenligningstabell

Egenskap	Abstrakt Klasse	Interface
Nøkkelord	<code>abstract class</code>	<code>interface</code>
Variabler	Kan ha alle typer (private, protected, etc.)	Kun konstanter (<code>static final</code>)
Arv	En klasse kan kun <code>extend</code> én	En klasse kan <code>implement</code> mange
Konstruktør	Kan ha konstruktør	Kan ikke ha konstruktør



Eksporter til Regneark



Et generelt eksempel

Tenk deg at du bygger et system for kjøretøy (helt uavhengig av oppgaven din):

- Du bruker en **abstrakt klasse** `Vehicle` for å definere hva et kjøretøy er. Den kan holde på felles data som `engineSize` og `fuelLevel`.
- Du bruker et **interface** `GPS` for å definere en *egenskap* eller *oppførsel*. Både en bil, en mobiltelefon og en båt kan implementere `GPS`, selv om de ikke er i samme klassehierarki.

For at jeg skal kunne hjelpe deg videre i forhold til oppgaven din:

Hvilke tanker har du gjort deg om disse konseptene i koden din så langt, og er det en spesifikk situasjon i oppgaven hvor du er usikker på hva du bør velge?

er skjermbilder):

Episode

JAWA

Movie

JAWA

Person

JAWA

Produk

JAWA

"Det er klasser vi har laget som det ikke er naturlig å lage konkrete objekter av. Hvilke er dette? Gjør disse klassene abstrakte. Test at alt fortsatt fungerer etter dette."

klassene TVSeries, episode, movie og role abstraherte jeg til production og person uten å omgjøre dem til abstract class ved å flytte felles instansvariabler og metoder. så jeg kan bare omgjøre production og person til abstract-class. tanker om hvordan det kan gjøres?



Programmering 2 Chat-Assistent • Egendefinert Gem

Analyse ▾

Hei! Det er flott å se at du allerede har gjort et godt forarbeid med å flytte felles variabler og metoder opp i superklassene – dette er selve kjernen i god objektorientert design.

Når oppgaven spør hvilke klasser det ikke er "naturlig å lage konkrete objekter av", handler det om å identifisere de klassene som fungerer som generelle maler fremfor spesifikke ting.

Hvilke bør være abstrakte?

Du nevner `Production` og `Person`. La oss se på dem:

- `Production`: Lager man noen gang en "produksjon" som ikke enten er en film eller en episode? Sannsynligvis ikke. `Production` er et generelt begrep, mens `Movie` og `Episode` er de konkrete tingene vi faktisk ser på. Her er `abstract` helt riktig.
- `Person`: Her må man tenke seg om. I ditt nåværende system, bruker du `Person` direkte (f.eks. for regissører), eller har du egne subclasser for `Director` eller `Actor`? Hvis du alltid bare bruker klassen `Person` for å representere et menneske, bør den forbli konkret. Hvis du derimot hadde hatt `class Director extends Person`, kunne `Person` vært abstrakt. Ut fra koden din ser det ut til at `Person` bør forbli slik den er.

Hvordan gjør du det i praksis?

For å gjøre en klasse abstrakt, trenger du bare å legge til nøkkelordet `abstract` i klassedefinisjonen.

Slik gjør du det med `Production`:

Java



```
public abstract class Production {  
    // Resten av koden din forblir lik  
}
```

Når du har gjort dette, vil Java nekte deg å skrive `new Production(...)` i `Main`. Du kan derimot fortsatt skrive `new Movie(...)` eller `new Episode(...)`.

Noen ting du bør sjekke etter endringen:

- Konstruktører:** En abstrakt klasse kan (og bør ofte) ha konstruktører. Disse kalles via `super()` fra subclassene (`Movie` og `Episode`), akkurat slik du allerede gjør i koden din.
- Abstrakte metoder:** Er det noen metoder alle produksjoner *må* ha, men som implementeres forskjellig? For eksempel, hvis `toString()` SKAL være unik for hver type, kan du markere den som `abstract` i `Production`.
- Testingen:** Siden du allerede bruker `Movie` og `Episode` i `Main.java`, vil koden din sannsynligvis kompilere og kjøre med en gang du legger til `abstract` på `Production`.

Et lite tips til koden din:

I `TVSeries.java` ser jeg at den arver fra `Production`:

```
public class TVSeries extends Production {
```

Er en TV-serie en "Production" på samme linje som en film, eller er en TV-serie heller en *samling* av produksjoner (episoder)? Hvis du ser på variablene i `Production` (som `runtime` og `director`), passer disse på en hel serie, eller hører de mest hjemme på de enkelte episodene? Dette er en fin nøtt å tenke på når man designer klassehierarkier!

Hvilke av disse klassene mener du er mest naturlig å låse for instansiering nå som du ser på sammenhengen?



da tenker jeg å abstrahere person klassen og lage director-klasse

Programmering 2 Chat-Assistent - Egendefinert Gerni

Det er en utmerket plan! Ved å gjøre `Person` abstrakt og innføre en `Director`-klasse, følger du prinsippet om at superklasser ofte representerer generelle konsepter som vi ikke ønsker å instansiere direkte.

Her er noen tanker om hvordan du kan gå frem for å implementere dette:

1. Gjøre `Person` abstrakt

Når du endrer `public class Person` til `public abstract class Person`, forteller du Java at dette er en ufullstendig mål. Siden du har felt som `firstName`, `lastName` og `age` her, vil disse automatisk bli arvet av alle subclasser du lager (som `Director` eller `Actor`).

2. Opprette `Director`-klassen

Siden `Director` nå skal være en spesifikk type person, må den arve fra `Person`.

- **Konstruktør:** Du må lage en konstruktør i `Director` som kaller `super(...)` for å gjenbruke logikken du allerede har i `Person`.
- **Spesialisering:** Er det noe en regissør har som en vanlig person ikke har? Kanskje en liste over filmer de har regissert, eller en spesifikk prislisse? Hvis ikke, fungerer klassen fortsatt som en viktig type-markør i koden din.

3. Oppdatering av de andre klassene

Dette er det viktigste steget for at endringen skal gi mening i koden din:

- I `Production`: Du har i dag et felt `private Person director`. Nå som du har en egen klasse for dette, bør du vurdere å endre datatypen til `private Director director`. Dette gjør koden mer tydelig.
- I `Episode` og `Movie`: Sjekk konstruktørene dine. I `Movie.java` tar du imot en `Person director`. Denne bør nå endres til `Director director` for å matche den nye strukturen.

Et generelt eksempel på utvidelse

Hvis man har en abstrakt klasse for et transportmiddel, kan man ha subclasser som krever ulike sertifikater:

Java



```
// Generell mål som ikke kan instansieres
public abstract class BaseUser {
    private String username;
    // Konstruktør og metoder...
}

// Spesifikk implementasjon
public class Admin extends BaseUser {
    private int accessLevel;

    public Admin(String username, int accessLevel) {
        super(username); // Kaller konstruktøren til BaseUser
        this.accessLevel = accessLevel;
    }
}
```

Velen videre

Når du har opprettet `Director`, må du også huske å oppdatere `Main.java`. Der du tidligere skrev `new Person("Walter", "White", 55)`, må du nå bestemme om Walter skal være en `Director` eller kanskje en `Actor` (hvis du lager den klassen også).

Et spørsmål til refleksjon:

Hvis du endrer `Person` til å være abstrakt, hva gjør du med de personene i `Main.java` som i dag fungerer som skuespillere (de du legger inn i `Role`)? Bør du også lage en `Actor`-klasse som arver fra `Person`?

Vil du at jeg skal se over hvordan du ser for deg at `Director`-klassen skal se ut før du begynner å endre i `Production`?

Main
JAVA

Actor
JAVA

Director
JAVA

Episode
JAO

får denne feilmelding:

▼

"Exception in thread "main"
java.lang.IllegalArgumentException: bound must be greater than origin
at java.base/...

Programmering 2 Chat-Assistent • Egendefinert Gam

Analysis ▼

Feilmeldingen `java.lang.IllegalArgumentException: bound must be greater than origin` betyr at du prøver å generere et tilfeldig tall der det øverste tallet (bound) er mindre enn eller lik det nederste tallet (origin).

I Java må `nextInt(origin, bound)` alltid ha en `bound` som er større enn `origin`.

Hvor ligger feilen?

Basert på feilmeldingen din skjer dette på linje 60 i `Main.java`:

Java

```
uniqueRole.add(listOfRoles.get(random.nextInt(rand1, rand2)));
```

Her bruker du `rand1` og `rand2` som grenser. Problemet oppstår på grunn av logikken rett over (linje 58):

Java

```
if (rand1 < rand2) {  
    for (i = rand1; i <= rand2; i++) {  
        uniqueRole.add(listOfRoles.get(random.nextInt(rand1, rand2))); // Her!  
    }  
}
```

Selv om du har sjekket at `rand1 < rand2`, er det to ting som går galt her:

- Løkken går for langt:** Du bruker `i <= rand2`. Hvis `rand1` er 2 og `rand2` er 3, vil løkken kjøre for `i=2` og `i=3`. Men i `random.nextInt(2, 3)` er den øvre grensen eksklusiv. Det betyr at det eneste mulige tallet den kan trekke er 2.
- Tomt intervall:** Hvis `rand2` er nøyaktig `rand1 + 1` (f.eks. 2 og 3), vil `nextInt(2, 3)` fungere. Men hvis `rand1` og `rand2` blir like eller har en logikk som gjør at de havner "oppå hverandre" i andre deler av koden, krasjer det.

Hvordan fikse det?

For å trekke en tilfeldig rolle fra hele listen din, bør du ikke bruke `rand1` og `rand2` som grenser inne i `nextInt()`. Du vil vel egentlig bare trekke en helt tilfeldig rolle fra `listOfRoles`?

Prøv å endre linje 60 (og tilsvarende i `else`-blokken) til:

Java

```
uniqueRole.add(listOfRoles.get(random.nextInt(0, listOfRoles.size())));
```

Et pedagogisk eksempel på feilen

Her er et generelt eksempel som viser hvorfor koden krasjer:

Java

```
Random r = new Random();  
int start = 5;  
int slutt = 5;  
  
// Dette vil kaste "IllegalArgumentException" fordi det ikke finnes  
// noen tall mellom 5 og 5 (når 5 er ekskludert).  
int tall = r.nextInt(start, slutt);
```

Et lite tips om designet ditt:

Jeg ser at `Role` arver fra `Actor`:

```
public class Role extends Actor {
```

I objektorientert programmering betyr `extends` at en rolle **er** en skuespiller. Er en rolle (f.eks. "Tony Stark") egentlig en skuespiller (Robert Downey Jr.), eller er det heller slik at en rolle **spilles** av en skuespiller? Siden du allerede har et felt `private Actor actor` inne i `Role`, har du både arv ("is-a") og komposisjon ("has-a") samtidig.

Spørsmål til deg:

Trenger `Role` egentlig å arve fra `Actor` når den allerede inneholder et `Actor`-objekt? Hva skjer hvis du fjerner `extends Actor` fra `Role`-klassen?

Vil du prøve å endre `nextInt`-logikken først og se om feilmeldingen forsvinner?

```
int rand1 = random.nextInt(0, personArrayList.size()+1);
int rand2 = random.nextInt(0, personArrayList.size()+1);
HashSet<Role> uniqueRole = new HashSet<>();

if (rand1<rand2){
    for (i = rand1; i <= rand2; i++){
        uniqueRole.add(listOfRoles.get(random.nextInt(rand1,rand2)));
    }
    ep.addRoles(new ArrayList<>(uniqueRole));
} else if (rand1==rand2){
    rand1 = random.nextInt(0, personArrayList.size()+1);
    rand2 = random.nextInt(0, personArrayList.size()+1);
} else {
    for (i = rand2; i <= rand1; i++){
        uniqueRole.add(listOfRoles.get(random.nextInt(rand2,rand1)));
    }
    ep.addRoles(new ArrayList<>(uniqueRole));
}
}

//hvordan starte if-testen på nytt
```

Programming 2 Chat-Assistent • Eigendefiniert Gem

For å starte en test på nytt i en slik situasjon, er den vanligste metoden å bruke en **løkke** (for eksempel en **while**-løkke) i stedet for bare en **if**. En løkke vil fortsette å kjøre helt til en bestemt betingelse er møtt – i ditt tilfelle at **rand1** og **rand2** ikke er like.

Her er hvordan du kan strukturere det:

1. Bruk en `while`-løkke for å garantere unike tall

I stedet for å prøve på nytt én gang med en `if`, kan du be Java fortsette å trekke tall helt til de er forskjellige:

Java

```
int rand1 = random.nextInt(0, personArrayList.size());
int rand2 = random.nextInt(0, personArrayList.size());

// Så lenge de er like, trekk rand2 på nytt
while (rand1 == rand2) {
    rand2 = random.nextInt(0, personArrayList.size());
}

// Nå er du GARANTERT at rand1 != rand2, og du kan fortsette med logikken din
if (rand1 < rand2) {
    // ... din kode
} else {
    // ... din kode
}
```

2. Hvorfor kræjer det fortsatt?

Selv om du fikser `rand1 == rand2`, har du en utfordring til i logikken din på linje 60 og 67 i `Main.java`.

```
Du skriver: uniqueRole.add(listOfRoles.get(random.nextInt(rand1, rand2)));
```

Hvis `rand1` er 5 og `rand2` er 6, vil `random.nextInt(5, 6)` fungere (det gir alltid 5). Men hvis `rand2` er større enn størrelsen på `listOfRoles`, vil du få en `IndexOutOfBoundsException`.

Et pedagogisk tips: Hvorfor bruker du `rand1` og `rand2` som grenser for å hente ut roller? Hvis målet er å legge til et tilfeldig antall roller til en episode, kan du vurdere denne enklere tilnærmingen:

1. Bestem et tilfeldig antall roller episoden skal ha (f.eks. mellom 1 og 5).
2. Kjør en `for`-løkke så mange ganger.
3. I hver runde trekker du en helt tilfeldig indeks fra `listOfRoles` (fra 0 til `listOfRoles.size()`).

3. En liten observasjon i koden din

Jeg ser i `Main.java` at du bruker `personArrayList.size() + 1` som grense i `nextInt()`. Husk at `personArrayList.get(index)` vil krasje hvis du trekker et tall som er lik `size()`, fordi indekser starter på 0. Det tryggeste er alltid å bruke `random.nextInt(8, personArrayList.size())`.

Hvordan tenker du at logikken med `rand1` og `rand2` skal fungere for antall roller? Er målet at de skal representere et "snitt" av listen, eller bare et tilfeldig antall?

men den skal kunne trekke tallet lik `size()`

Programmering 2 Chat-Assistent • Egendefinert Gem

Hvis du vil at `nextInt` skal kunne trekke et tall som er lik `size()`, må du være veldig forsiktig med hvordan du bruker dette tallet senere.

I Java (og de fleste programmeringsspråk) er lister 0-indekserte. Det betyr at hvis en liste har `size() = 5`, så er de gyldige indeksene 0, 1, 2, 3 og 4. Hvis du prøver å kalle `.get(5)`, vil programmet krasje med en `IndexOutOfBoundsException`.

Her er forklaringen på hvordan du kan løse logikken din:

1. Hvorfor `size() + 1` skaper problemer

Når du skriver `random.nextInt(0, personArrayList.size() + 1)`, betyr det at `rand1` eller `rand2` faktisk kan bli lik verdien til `size()`.

- Hvis du bruker disse tallene som indekser i en liste (f.eks. `listOfRoles.get(rand1)`), vil koden krasje hvis tallet er lik listeprisen.
- Hvis målet ditt bare er å bruke tallene som grenser for en løkke, fungerer det fint, men du må sikre at du aldri bruker dem til å hente elementer direkte hvis de er utenfor rekkevidde.

2. Hvordan starte "på nytt" med en løkke

For å tvinge programmet til å trekke nye tall helt til de er ulike (slik at `rand1 < rand2` eller `rand1 > rand2` alltid er sant), bør du bruke en `do-while`-løkke. Den kjører koden én gang, og gjentar den så lenge betingelsen er sann.

Slik kan du skrive om blokken din:

Java

```
int rand1, rand2;

// Denne løkken kjører helt til rand1 og rand2 er forskjellige
do {
    rand1 = random.nextInt(0, personArrayList.size() + 1);
    rand2 = random.nextInt(0, personArrayList.size() + 1);
} while (rand1 == rand2);

HashSet<Role> uniqueRole = new HashSet<>();

if (rand1 < rand2) {
    for (i = rand1; i <= rand2; i++) {
        // VIKTIG: Her må du sikre at du ikke trekker en indeks som er >= listOfRoles.size()
        uniqueRole.add(listOfRoles.get(random.nextInt(0, listOfRoles.size())));
    }
} else {
    for (i = rand2; i <= rand1; i++) {
        uniqueRole.add(listOfRoles.get(random.nextInt(0, listOfRoles.size())));
    }
}

ep.addRoles(new ArrayList<>(uniqueRole));
```

3. Oppsummering av feilkilder i `Main.java`

- Bound-feilen:** `random.nextInt(start, slutt)` kaster feil hvis `start == slutt`. Ved å bruke `do-while` over, slipper du `else if (rand1 == rand2)`-logikken som ikke fungerte.
- Indeks-feil:** Selv om du vil trekke tallet `size()`, må du huske at `listOfRoles.get(...)` kun godtar tall fra 0 til `size() - 1`.

Et lite tips til designet ditt:

Jeg ser at `Role` arver fra `Actor`. I objektorientert programmering bør `extends` brukes når noe er en type av noe annet. Er en "Rolle" (som "Tony Stark") virkelig en "Skuespiller", eller er det mer naturlig at rollen har en skuespiller tilknyttet seg? Du har allerede begge deler i `Role.java` (både arv og et felt for `Actor`).

```

int rand1 = random.nextInt(0, listOfRoles.size()+1);
int rand2 = random.nextInt(0, listOfRoles.size()+1);
while (rand1==rand2) {
    rand1 = random.nextInt(0, listOfRoles.size()+1);
    rand2 = random.nextInt(0, listOfRoles.size()+1);
}
HashSet<Role> uniqueRole = new HashSet<>();
if (rand1<rand2){
    for (i = rand1; i <= rand2;i++){
        uniqueRole.add(listOfRoles.get(random.nextInt(rand1,rand2)));
    }
    ep.addRoles(new ArrayList<>(uniqueRole));
}else {
    for (i = rand2; i <= rand1;i++){
        uniqueRole.add(listOfRoles.get(random.nextInt(rand2,rand1)));
    }
    ep.addRoles(new ArrayList<>(uniqueRole));
}
}
får denne feilen: "Exception in thread "main"
java.lang.IndexOutOfBoundsException: Index 4 out of
bounds for length 4
    at java.base/
jdk.internal.util.Preconditions.outOfBounds(Preconditions.j
ava:100)
    at java.base/
jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Pr
econditions.java:106)
    at java.base/
jdk.internal.util.Preconditions.checkIndex(Preconditions.ja
va:302)
    at java.base/
java.util.Objects.checkIndex(Objects.java:365)
    at java.base/java.util.ArrayList.get(ArrayList.java:428)
    at oblig4.Main.main(Main.java:59)" er det på grunn av
size()-1 problemet?

```


Ja, det stemmer helt! Du har truffet spikeren på hodet. Feilmeldingen `Index 4 out of bounds` for `length 4` betyr at du prøver å hente noe fra plass nummer 4, men i en liste med 4 elementer er de gyldige plassene bare 0, 1, 2 og 3.

Det er spesielt to steder i koden din som forårsaker dette nå:

1. `size() + 1` i grensene dine

Når du skriver `random.nextInt(0, listOfRoles.size() + 1)`, så tillater du at `rand1` eller `rand2` blir lik selve størrelsen på listen.

Hvis `listOfRoles.size()` er 4, kan `rand1` bli 4. Når du senere kjører løkken `for (i = rand1; i <= rand2; i++)`, og bruker `i` eller disse grensene til å hente fra listen, vil koden krasje.

2. Logikken inne i `get()`

I løkken din skriver du:

```
uniqueRole.add(listOfRoles.get(random.nextInt(rand1, rand2)));
```

Her er det to potensielle problemer:

- Hvis `rand2` har blitt trukket til å være `size()`, vil `random.nextInt(rand1, rand2)` kunne returnere en verdi som er akkurat for høy for listen.
- Du bruker `rand1` og `rand2` som om de er gyldige indekser i `listOfRoles`, men de er basert på `personArrayList` i din forrige melding. Det er viktig at grensene du bruker i `nextInt` alltid samsvarer med størrelsen på den listen du faktisk skal hente fra.

Hvordan løse det pedagogisk?

Hvis målet ditt er å legge til et tilfeldig utvalg av roller til en episode, er det tryggere å skille mellom hvor mange roller du vil ha, og hvilken indeks du henter fra.

Prøv å tenke slik:

1. **Antall:** Bestem hvor mange roller som skal legges til (f.eks. mellom `rand1` og `rand2`).
2. **Henting:** Når du skal hente selve rollen med `.get()`, bruk alltid `0` som start og `listOfRoles.size()` som (eksklusiv) slutt. Da er du garantert å aldri treffe utenfor.

Et generelt eksempel på trygg henting fra liste:

Java

```
// Dette vil ALDRI krasje uansett størrelse på listen
int tilfeldigIndeks = random.nextInt(0, minListe.size());
String objekt = minListe.get(tilfeldigIndeks);
```

Et spørsmål om koden din i `Main.java`:

Jeg ser i filene dine at `Role` nå arver fra `Actor` (`public class Role extends Actor`).

Inne i `Role`-konstruktøren kaller du `super(roleFirstName, roleLastName, age)`.

Dette betyr at når du lager en rolle som "Tony Stark", så lager du teknisk sett en skuespiller med navnet "Tony Stark". Men så sender du inn et *annet* `Actor`-objekt (f.eks. Robert Downey Jr.) i tillegg.

Tenk over dette: Når du printer ut rollen, vil du at den skal bruke navnet til karakteren ("Tony Stark") eller navnet til skuespilleren? Slik koden er nå, lagrer du to sett med navn og alder for hver rolle. Ville det vært enklere om `Role` ikke arvet fra `Actor`, men bare *inneholdt* en skuespiller?

Fikk du rettet opp i `size()`-problemet i `Main.java` slik at krasjen forsvant?



"Henting: Når du skal hente selve rollen med `.get()`, bruk alltid 0 som start og `listOfRoles.size()` som (eksklusiv) slutt. Da er du garantert å aldri treffe utenfor." men jeg vil at trekkingen skal starte fra f.eks. 1 eller 2 og kunne slutte på tallet lik `size()`

Programmering 2 Chat-Assistent • Egendefinert Gem

Skjønner! Hvis du ønsker at utvalget skal starte fra en bestemt indeks (f.eks. 1 eller 2) og kunne gå helt opp til og med det siste elementet i listen, må vi justere grensene i `random.nextInt()` og hvordan du henter dem.

Her er de to viktigste tingene å huske på i Java for å få til dette:

1. **Indekser:** Siste element i listen har alltid indeks `size() - 1`.
2. **Ekklusiv øvre grense:** `random.nextInt(start, slutt)` trekker et tall fra og med `start`, til (men ikke med) `slutt`.

Løsningen for din logikk:

Hvis du vil at `rand2` skal kunne representere den siste indeksen i listen, og du har brukt `listOfRoles.size() + 1` som grense, kan `rand2` bli lik `size()`. Da må du passe på at du aldri bruker dette tallet direkte i en `.get()`-metode.

Her er hvordan du kan skrive det for at det skal fungere uten krasj:

Java



```
// Vi trekker grenser. rand2 kan nå bli lik size()
int rand1 = random.nextInt(0, listOfRoles.size() + 1);
int rand2 = random.nextInt(0, listOfRoles.size() + 1);

// ... (while-løkken din for å sikre at de er ulike) ...

if (rand1 < rand2) {
    for (i = rand1; i < rand2; i++) { // Bruk < i stedet for <= her
        // Nå kan i aldri bli lik rand2.
        // Hvis rand2 er 4 (størrelsen på listen), vil i stoppe på 3.
        uniqueRole.add(listOfRoles.get(i));
    }
}
```

Hvorfør løser dette "Index 4 out of bounds"?

Hvis listen din har 4 elementer (indeks 0, 1, 2, 3):

- `listOfRoles.size()` er 4.
- Hvis `rand2` blir 4.
- Løkken `for (i = rand1; i < 4; i++)` vil kjøre for siste gang når `i = 3`.
- `listOfRoles.get(3)` er det siste lovlege elementet.
- Når `i` blir 4, er ikke lenger `4 < 4` sant, og løkken stopper før den prøver å hente indeks 4.

Vil du trekke helt tilfeldige roller innenfor intervallet?

I koden din hadde du: `uniqueRole.add(listOfRoles.get(random.nextInt(rand1, rand2)));`.

Hvis du vil fortsette med dette, men unngå krasj, må du sørge for at `rand2` aldri er større enn `listOfRoles.size()`.

Prøv denne justeringen:

Java



```
// Sett grensene slik at de treffer nøyaktig på indeksene
int rand1 = random.nextInt(0, listOfRoles.size()); // Kan bli 0 til size-1
int rand2 = random.nextInt(0, listOfRoles.size() + 1); // Kan bli 0 til size

// Innen i for-løkken din:
uniqueRole.add(listOfRoles.get(random.nextInt(rand1, rand2)));
```

Når `random.nextInt(rand1, rand2)` kjører, vil det største tallet den kan lage være `rand2 - 1`.

Hvis `rand2` er `size()`, vil det største tallet bli `size() - 1`, som er den siste lovlege plassen i listen!

Et lite opptøyingespørsmål til designet ditt:

Jeg ser at `Role` nå arver fra `Actor`. Siden du nå har gjort `Person` abstrakt, har du fått et fint hierarki! Men i `Role.java` ser jeg at du både har `extends Actor` og et felt `private Actor actor`.

Dersom du fjerner `extends Actor` fra `Role`, vil du fortsatt kunne beholde all funksjonalitet ved å bruke `actor`-objektet du har lagret i klassen. Hva tenker du er fordelene med at en `Role` er en `Actor` i ditt program?

Actor
JAVA

Director
JAVA

Episode
JAVA

Movie
JAVA



Tilbake til første spørsmål, mangler jeg noe som kan flyttes til abstract-klassene

Programmering 2 Chat-Assistent • Egendefinert Gem

Når vi ser på koden din nå som du har innført både `Production` og `Person` som abstrakte klasser, har du kommet langt med å organisere logikken. Men det er fortsatt et par steder hvor du kan flytte funksjonalitet "opp" for å gjøre koden enda mer elegant og fjerne duplisering.

Her er de viktigste punktene:

1. `toString()` i `Production` vs `Movie / Episode`

I `Production.java` har du en fin `toString()`. Men både `Movie` og `Episode` overstyrer (`@Override`) denne helt.

Tip: Du kan bruke `super.toString()` inne i subclassenes `toString()` for å hente den generelle infoen (tittel, beskrivelse, dato), og så bare legge til det som er spesifikt for filmen eller episoden etterpå.

2. Roller og "Cast"

I dag har du `ArrayList<Role> roles` i `Production`. Dette er bra! Men:

- `Movie` har en lenke i sin `toString()` for å skrive ut roller.
- `TVSeries` (som også er en `Production`) har en metode `getCast()` som samler alle roller fra alle episoder.

Siden alle produksjoner har roller, kan du vurdere om metoder for å formatere eller håndtere rollelisten (f.eks. en `getRolesSummary()`) kan ligge i `Production`. Da slipper du å skrive nesten lik kode inne i `Movie` og `Episode`.

3. `Director` i `Production`

Du har flyttet `Director director` opp i `Production`. Det er veldig bra! Men sjekk konstruktørene dine:

- I `Production.java` har du konstruktører som tar imot `Director`.
- I `Episode.java` tar den ene konstruktøren imot `Person director`. Siden `Person` nå er abstrakt og du har en egen `Director`-klasse, bør denne parameteren endres til `Director director` for å være konsistent med superklassen.

4. Spesielltilfellet `Role`

Dette er kanskje det viktigste punktet for "ren kode" akkurat nå. I `Role.java` har du:

```
public class Role extends Person
```

Dette betyr at en `Role` er en `Person`. Men inne i klassen har du også:

```
private Actor actor;
```

Dette skaper en dobbel lagring av informasjon. En "Rolle" har nå både et navn via arven fra `Person` (karakterens navn) og et navn via `Actor`-objektet (skuespillerens navn).

Mitt forslag:

I stedet for at `Role` arver fra `Person`, kan den forbli en vanlig klasse som inneholder to ting:

1. Karakternavnet (String)
2. Skuespilleren (Actor-objekt)

Dette er ofte ryddigere enn å bruke arv for noe som ikke er en "ekte" person i systemet ditt, men bare en kobling mellom en karakter og en skuespiller.

Oppsummering av hva du kan gjøre nå:

- Endre parameteren i `Episode`-konstruktøren fra `Person` til `Director`.
- Se om du kan bruke `super.toString()` i `Movie` for å slippe å skrive `getTitle()` og `getDescription()` på nytt der.
- Vurder om `Role` egentlig trenger å være en `Person` (arv).

Hva tenker du om `Role`-klassen? Virker det logisk at den arver fra `Person`, eller føles det mer som en kobling mellom en karakter og en skuespiller?

Main

❌ JAVA

Actor

❌ JAVA

Director

❌ JAVA

Episode

❌ JAVA

hva med nå?

Programmering 2 Chat-Assistent • Egendefinert Gern

ⓘ Dette er bare ment som informasjon. Kontakt lege hvis du trenger medisinske råd eller diagnoser.

Nå ser koden din mye mer profesjonell og strukturert ut! Du har gjort gode valg ved å gjøre `Person` og `Production` abstrakte, og du har ryddet opp i bruken av `super.toString()`.

Her er en vurdering av hva som er bra nå, og de siste små detaljene du kan vurdere å flytte eller justere:

Dette er veldig bra nå:

- **Abstrakte klasser:** `Person` og `Production` er nå `abstract`. Det er helt korrekt, da vi ikke vil lage en "generell" produksjon, men heller en spesifikk film eller episode.
- **Bruk av `super.toString()`:** I både `Movie` og `Episode` bruker du nå `super.toString() + sb`. Dette er skolebokeneksempel på god gjenbruk av kode!
- **Type-sikkerhet:** Du har endret `director` i `Production` fra `Person` til `Director`. Dette gjør at du nå er sikker på at en produksjon faktisk har en regissør, ikke bare en tilfeldig person.

Ting du fortsatt kan nytte til de abstrakte superklassene:

1. Logikken for Roller (i `Production`):

Akkurat nå har du `private ArrayList<Role> roles` i `Production`. Det er bra. Men i `Movie.java` har du en lenke i `toString()` som skriver ut alle rollene.

- **Forslag:** Flytt metoden for å printe rollelisten til `Production.java`. Du kan for eksempel lage en metode `public String getRolesAsString()` i `Production`. Da kan både `Movie` og andre fremtidige subclasser bruke denne uten å skrive lenken på nytt.

2. Cast-logikken (fra `TVSeries` til `Production` ?):

`TVSeries` har en veldig nyttig metode: `getCast()`, som samler alle unike roller.

- **Vurdering:** Siden en `Movie` også har en cast (liste med roller), kunne du vurdert å definere `getCast()` i `Production`. For en `Movie` vil den bare returnere `roles`-listen, mens for `TVSeries` må den (som du har gjort) gå gjennom alle episoder. Ved å ha metoden i `Production`, kan du behandle alle produksjoner likt når du vil vite hvem som er med.

En viktig logisk detalj i `Role.java`:

Du har nå: `public class Role extends Actor`.

Inne i klassen har du også: `private Actor actor;`

Dette er fortsatt litt "smør på flesk" (redundant). Siden `Role` arver fra `Actor`, har en `Role`-instans allerede et fornavn, et etternavn og en alder. Når du i tillegg har et `Actor`-objekt inni, lagrer du informasjonen to ganger.

Slik ser objektet ut i minnet nå:

- `Role`-objektet (har navn/alder fordi det er en `Actor/Person`)
- `Actor`-objektet inni (har navn/alder på nytt)

Anbefaling:

Gjør `Role` til en vanlig klasse (fjern `extends Actor`). La den kun inneholde:

1. `String roleName` (karakterens navn i filmen)
2. `Actor actor` (objektet som representerer skuespilleren)

Dette skiller tydelig mellom rollen (f.eks. Batman) og mennesket (f.eks. Christian Bale).

Oppsummering - mangler du noe?

Egentlig ikke! Du har dekket de viktigste kravene for abstraksjon og arv. Det eneste som gjenstår er "polering" av arkitekturen (som `Role`-eksempelet over) for å unngå at du lagrer samme data flere steder.

Er du fornøyd med hvordan `Main.java` kjører nå etter at vi fikset `Random`-krasjen, eller er det fortsatt noe som oppfører seg merkelig?



```
public class Role{
    private String roleFirstName;
    private String roleLastName;
    private Actor actor;

    public Role(String roleFirstName, String roleLastName,
        Actor actor){
        setRoleFirstName(roleFirstName);
        setRoleLastName(roleLastName);
        setActor(actor);
    }

    @Override public String toString() {
        return String.format("%s %s (%s)", getRoleFirstName(),
            getRoleLastName(), getActor().getFullName());
    }

    public String getRoleFirstName() {return roleFirstName;}
    public void setRoleFirstName(String roleFirstName)
    {this.roleFirstName = roleFirstName;}
    public String getRoleLastName() {return roleLastName;}
    public void setRoleLastName(String roleLastName)
    {this.roleLastName = roleLastName;}
    public Person getActor() {return actor;}
    public void setActor(Actor actor) {this.actor = actor;}
} sånn?
```

Programmering 2 Chat-Assistent • Egendefinert Gem

Ja, akkurat slik! Dette er en mye ryddigere måte å løse det på.

Ved å fjerne arven (`extends Actor`) og heller bruke **komposisjon** (at en `Role` har en `Actor`), har du nå et tydelig skille mellom karakteren i filmen og skuespilleren i det virkelige liv.

Hvorfor er dette bedre?

1. **Ingen duplisering:** Du slipper å lagre alder og navn to ganger for samme person.
2. **Logisk riktig:** En "Rolle" er ikke et menneske (Person), men en beskrivelse av en karakter som spilles av et menneske.
3. **Flersidighet:** Hvis du senere vil legge til mer info om selve karakteren (f.eks. `isHero` eller `superpowers`), blander du ikke dette sammen med skuespillerens private info.

En liten detalj for å gjøre koden helt perfekt:

Jeg ser at du har endret `getActor()` til å returnere en `Person` :

Java



```
public Person getActor() {return actor;}
```

Siden variabelen din er deklartert som `private Actor actor;` , er det mest naturlig at metoden returnerer en `Actor` . Da har du tilgang til alt som er spesifikt for skuespillere (hvis du legger til noe der senere).

Et lite tips til `toString()` :

Nå som du har ryddet i `Role` , ser `Movie.java` sin `toString()` også veldig bra ut fordi den bare kan kalle `role.toString()` og få ut akkurat det som trengs.

Oppsummering av strukturen din nå:

Du har nå et veldig sterkt objektorientert design:

- `Person` (Abstract): Grunnlag for alle mennesker.
- `Actor` & `Director` : Spesialiserte typer mennesker.
- `Production` (Abstract): Grunnlag for alt som kan sees på skjerm.
- `Movie` & `Episode` : Spesifikke produksjoner.
- `Role` : Koblingen (linet) mellom en karakter og en skuespiller.

Er det noe spesiell du føler er vanskelig med `TVSeries` sin `getCast()`-metode nå, eller fungerer den som den skal med de nye `Role`-objektene?

Oppgave 2.2 (brukte KI og W3Schools som hjelpemidler):

Actor

JAVA

Director

JAVA

Episode

JAVA

Movie

JAVA



"Definer et interface med navn TVSeriesRepository.

Dette interfacet skal fungere som en "kontrakt" på hvilke metoder som

må implementeres i forbindelse med lagring og uthenting av TVSeries-objekter. Interfacet skal altså kunne bli...

Programmering 2 Chat-Assistent • Egendefinert Gem

Analyse

Hei! Det er kjempebra at du har kommet i gang med interfacet. Et interface er nettopp en slik "kontrakt" som definerer *hva* som skal gjøres, uten å bestemme *hvordan* det skal gjøres. Når jeg ser på koden din i `TVSeriesRepository.java`, er du veldig nære målet, men det er et par viktige detaljer i oppgaveteksten som ikke stemmer helt overens med signaturene dine.

Noen punkter du bør vurdere:

1. Returtyper:

- I oppgaven står det at `getAllTVSeries()` skal returnere en `ArrayList` med alle lagrede objekter. Akkurat nå er metoden din definert som `void`. Hvilken datatype bør stå der i stedet for `void`?
- Det samme gjelder `getTVSeriesByTitle(String title)`. Oppgaven sier den skal returnere et `TVSeries`-objekt. Hva bør returtypen være her?

2. Bruk av `public` og `abstract`:

- I et interface i Java er alle metoder som standard `public` og `abstract`. Det er ikke *feil* å skrive det, men det er ofte ansett som god skikk å utelate disse nøkkelordene inne i selve interfacet for å holde koden renere.

3. Imports:

- Siden du bruker `ArrayList` og `TVSeries` i metodene, må du sørge for at disse er tilgjengelige. Du har allerede `import java.util.ArrayList;`, så det ser bra ut!

Et lite tankeeksperiment

For å illustrere konseptet uten å bruke din oppgave: Hvis vi hadde et interface for en bil-lagring, kunne det sett slik ut:

Java



```
public interface CarRepository {  
    // Tar imot en liste og lagrer (void er greit her da vi bare sender data inn)  
    void saveCars(ArrayList<Car> cars);  
  
    // Skal HENTE ut alle biler, derfor må den returnere listen  
    ArrayList<Car> getAllCars();  
}
```

Hvilke endringer tror du må til i dine metodesignaturer for at de faktisk skal returnere det oppgaven ber om?

Når du har sett på returtypene, er neste naturlige steg å tenke på hvilken klasse som skal implementere dette interfacet. Har du gjort deg noen tanker om det?

```
void addListOfTVSeries(ArrayList<TVSeries> listOfTVSeries);  
ArrayList<TVSeries> getAllTVSeries();  
TVSeries getTVSeriesByTitle(String title); sånn da?
```

Programmering 2 Chat-Assistent • Egendefinert Gem

Det ser helt korrekt ut! Nå følger metodene dine kontrakten som var spesifisert i oppgaveteksten:

- `addListOfTVSeries` : Har `void` som returtype fordi den kun skal motta og lagre data, ikke sende noe tilbake.
- `getAllTVSeries` : Returnerer nå en `ArrayList<TVSeries>`, slik at den som kaller på metoden faktisk får ut listen over serier.
- `getTVSeriesByTitle` : Returnerer et `TVSeries`-objekt, som gjør det mulig å hente ut én spesifikk serie basert på tittelen.

Et lite tips om god praksis i Java:

I et `interface` er alle metoder automatisk `public` og `abstract`. Selv om det ikke er feil å skrive det (slik du hadde i forrige versjon), velger de fleste å skrive dem slik du har gjort nå, for å holde koden så ren som mulig.

Veien videre

Nå som "kontrakten" (interfacet) er på plass, er neste steg å lage en klasse som faktisk overholder denne kontrakten. Oppgaven nevner ofte at man skal lage en klasse (for eksempel `TVSeriesJSONRepository` eller lignende) som `implements TVSeriesRepository`.

Har du tenkt på hvilken klasse som skal implementere dette interfacet, og hvordan du skal gå frem for å skrive logikken inne i de tre metodene?

Oppgave 2.3 (brukte KI og W3Schools som hjelpemidler.):

```
@Override public ArrayList<TVSeries> getAllTVSeries() {
    ArrayList<TVSeries> fetchedTVS = new ArrayList<>();
    try (BufferedReader bufferedReader = new
        BufferedReader(new FileReader(file))) {
        if (file.canRead()) {
            String line;
            while ((line = bufferedReader.readLine()) != null) {
                String[] values = line.split(",");
                String title = values[0];
                LocalDate releaseDate = LocalDate.parse(values[1]);
                String description = "";
                for (int i = 2; i < values.length; i++) {
                    description = values[i];
                }
                TVSeries newTVS = new TVSeries(title, description,
                    releaseDate);
                fetchedTVS.add(newTVS);
            }
        }
    } catch (FileNotFoundException exception) {
        System.err.println("Could not find file with name " +
            file.getName());
    } catch (IOException e) {
        System.err.println(e.getMessage());
    }
    return new ArrayList<>(fetchedTVS);
}
```

```
får denne feilmeldingen: Exception in thread "main"
java.time.format.DateTimeParseException: Text 'wawa'
could not be parsed at index 0
    at java.base/
java.time.format.DateTimeFormatter.parseResolved0(Dat
eTimeFormatter.java:2108)
    at java.base/
java.time.format.DateTimeFormatter.parse(DateTimeForm
atter.java:2010)
    at java.base/
java.time.LocalDate.parse(LocalDate.java:437)
    at java.base/
java.time.LocalDate.parse(LocalDate.java:422)
    at
oblig4.modules.TVSeriesCSVRepository.getAllTVSeries(T
VSeriesCSVRepository.java:35)
    at oblig4.CSVMain.main(CSVMain.java:18)
```

koden parser title istedenfor releasedate. hvorfor

Hei! Det er flott at du deler både koden og feilmeldingen. Det gjør det mye enklere å se hva som skjer.

Feilmeldingen `java.time.format.DateTimeParseException: Text 'wawa' could not be parsed` bekrefter mistanken din: Programmet prøver å tolke strengen "wawa" som en dato, noe den selvfølgelig ikke er.

La oss ta en kikk på hvordan du henter ut data fra `values`-arrayet ditt.

Hva skjer i koden din nå?

Når du kjører `line.split(":")`, deler Java opp linjen i biter basert på kolon. Hvis linjen i fila din for eksempel ser slik ut:

```
Tittel:2023-01-01:Beskrivelse
```

...vil `values` se slik ut:

- `values[0]` = "Tittel"
- `values[1]` = "2023-01-01"
- `values[2]` = "Beskrivelse"

I koden din henter du ut verdiene slik:

Java

```
String title = values[0];  
LocalDate releaseDate = LocalDate.parse(values[1]);
```

Siden feilmeldingen sier at den prøver å parse teksten "wawa" (merk mellomrommet foran), betyr det at på den spesifikke linjen i fila di, så ligger denne teksten i posisjon `values[1]`.

Spørsmål til refleksjon:

1. **Strukturen på fila:** Hvordan ser den faktiske linjen i CSV-fila di ut for den serien som feiler? Er du helt sikker på at rekkefølgen på kolonnene i fila samsvarer med indeksene (`[0]` , `[1]` , `[2]`) du bruker i koden?
2. **Skilletegn:** Bruker fila di faktisk kolon (`:`) som skilletegn på alle linjer, eller kan det være andre tegn der?
3. **Håndtering av tekst:** Ser du noe i feilmeldingen som tyder på at det er ekstra mellomrom i dataene dine? (Hint: Se på `' wawa '`). Hvordan kan man fjerne overflødige mellomrom fra en `String` i Java før man prøver å bruke den?

Et lite tips om kodekvalitet (valgfritt):

Du har en `for`-løkke som går gjennom `description`. Akkurat nå vil denne løkken overskrive `description`-variabelen for hver iterasjon, slik at det bare er den aller siste biten av linjen som blir lagret. Hvis beskrivelsen din kan inneholde kolon, må du kanskje tenke på en annen måte å sette sammen denne strengen på.

Hva tror du skjer med rekkefølgen i `values`-arrayet hvis en av linjene i fila di har et ekstra kolon et sted det ikke skal være?

Ta en sjekk på tekstfila di og se om rekkefølgen stemmer, så kan vi se videre på det etterpå!

så jeg må bare fjerne mellomrommet mellom : og wawa før den lagres

Programmering 2 Chat-Assistent • Egendefinert Gem

Det stemmer! Hvis filen din inneholder tekst som `Tittel : wawa : Beskrivelse`, vil `values[1]` inneholde strengen `" wawa"`. Når `LocalDate.parse()` ser det mellomrommet foran datoen, skjønner den ikke at det er en del av en dato, og kaster derfor en feilmelding.

I Java har `String`-klassen en veldig nyttig metode for å fjerne "whitespace" (mellomrom, tabulator, linjeskift) fra starten og slutten av en tekststreng.

Hvordan håndtere uønskede mellomrom?

Du kan bruke metoden `.trim()` på strengen før du sender den til parsing. Her er et generelt eksempel på hvordan dette fungerer:

Java



```
String inputWithSpaces = " 2024-12-24 ";
// .trim() fjerner mellomrommene før og etter
String cleanedInput = inputWithSpaces.trim();

// Nå vil parse-metoden fungere fordi den mottar "2024-12-24"
LocalDate date = LocalDate.parse(cleanedInput);
```

Noe å tenke på:

1. **Hvor bør du trimme?** Er det bare datoen som kan ha mellomrom, eller kan det være lurt å kjøre `.trim()` på `title` og `description` også for å sikre at dataene i objektene dine er "rene"?
2. **Robusthet:** Hva skjer hvis en linje i fila er helt tom, eller hvis noen har glemt å skrive inn datoen? Det kan være lurt å vurdere om koden din bør sjekke lengden på `values`-arrayet før du prøver å aksessere indeksene.

Et lite oppfølgingsspørsmål til deg:

Ser du nå hvorfor `description` i koden din kanskje ikke får med seg alt hvis det er flere kolon i beskrivelsen? (Tenk på hva som skjer i `for`-løkken din der du setter `description = values[1]`).

Vil du at vi skal se nærmere på hvordan du kan håndtere beskrivelsen mer effektivt, eller vil du prøve å implementere `.trim()` først?

men den ser slik ut:

Title: wawa
Release date: 11.9.2001
Description: ada kadabra sim sala binz

Title: nya
Release date: 22.2.2222
Description: neko

Programmering 2 Chat-Assistent • Egendefinert Gem

Takk for at du delte filformatet! Dette forklarer saken veldig godt.

Det viser seg at filen din ikke er en typisk CSV-fil (hvor hver linje er et nytt objekt), men snarere en tekstfil der et objekt går over flere linjer.

Hvorfor koden din teller nå

Koden din bruker `while ((line = bufferedReader.readLine()) != null)`. Dette betyr at Java leser **én og én linje**.

1. På den første linjen leser den: `"Title: wawa"`.
2. Den splitter på `:` og får `values[0] = "Title"` og `values[1] = "wawa"`.
3. Deretter prøver den å kjøre `LocalDate.parse("wawa")`.

Siden `"wawa"` ikke er en dato (det er jo tittelen!), krasjer programmet. Programmet ditt forventer at alt (tittel, dato og beskrivelse) finnes på én enkelt linje, men i fila di ligger de på tre forskjellige linjer.

Hvordan lese dette?

For å lese denne filen riktig, må du endre logikken slik at du leser tre linjer (og gjerne den tomme linjen mellom dem) før du lager et `TVSeries`-objekt.

Her er en logisk skisse på hvordan du kan gripe det an:

1. **Les tittel-linjen:** Hent ut verdien etter kolonet.
2. **Les dato-linjen:** Hent ut verdien etter kolonet.
3. **Les beskrivelse-linjen:** Hent ut verdien etter kolonet.
4. **Håndter dato-formatet:** Legg merke til at datoen i fila di er `11.9.2001`. Standard `LocalDate.parse()` forventer formatet `YYYY-MM-DD` (f.eks. `2001-09-11`).
5. **Opprett objektet:** Bruk de tre verdiene du har hentet ut til å kalle `new TVSeries(...)`.
6. **Hopp over tomrom:** Les den tomme linjen før du starter på neste serie.

En viktig merknad om datoer

Når datoen ikke følger ISO-standard, må du fortelle Java hvordan den skal tolkes ved hjelp av en `DateTimeFormatter`.

Eksempel på format-matching:

Hvis du har en dato som `24.12.2023`, må formatet defineres slik (uavhengig av oppgaven):

Java

```
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("d.M.yyyy");  
LocalDate date = LocalDate.parse("24.12.2023", formatter);
```

Valen videre

For å komme i mål, må du restrukturere `while`-løkken din. I stedet for å anta at alt er på én linje, må du tenke: "Hvordan kan jeg lese flere linjer inne i løkken for å fylle opp ett enkelt `TVSeries`-objekt?"

Hva tenker du er den enkleste måten å lese de neste to linjene på når du allerede er inne i `while`-løkken? (Et lite tips: Du kan kalle `bufferedReader.readLine()` flere ganger inne i løkken for å "bruke opp" de neste linjene i fila).

hvordan splitter jeg via linjeskift istedenfor :

Programmering 2 Chat-Assistent - Egendefinert Gem

For å splitte basert på linjeskift i Java, må vi først se på hvordan `BufferedReader` fungerer. Akkurat nå bruker du `readLine()`, som automatisk leser én linje av gangen og fjerner selve linjeskiftet for deg.

Siden filen din er strukturert over flere linjer per objekt, har du to hovedstrategier:

1. Fortsette med `readLine()` (Anbefalt)

I stedet for å splitte hele filen, kan du lese de linjene du trenger inne i løkken. Siden du vet at en serie består av tre linjer med data (og kanskje en tom linje), kan du gjøre slik:

Java

```
while ((line = bufferedReader.readLine()) != null) {
    if (line.isBlank()) continue; // Hopper over tomme linjer

    // 1. linje: Title: wawa
    String title = line.split(":")[1].trim();

    // 2. linje: Release date: 11.9.2001
    String dateLine = bufferedReader.readLine();
    String dateString = dateLine.split(":")[1].trim();

    // 3. linje: Description: ada kadabra
    String descLine = bufferedReader.readLine();
    String description = descLine.split(":")[1].trim();

    // Her kan du nå parse datoen og opprette objektet
}
```

2. Bruk `Scanner` med en annen "delimiter"

Hvis du vil lese inn alt og splitte manuelt, kan du bruke en `Scanner` og sette den til å splitte på doble linjeskift (som ofte skiller blokker med tekst).

Java

```
// \n er linjeskift i Unix, \r\n i Windows. \R dekker begge.
scanner.useDelimiter("\\R\\R");
```

Utfordringen med ditt filformat

Siden filen din ser slik ut:

Title: wawa

Release date: 11.9.2001

...må du håndtere to ting uansett hvilken metode du velger:

1. **Prefix-fjerning:** Du må fjerne "Title: " før du lagrer selve tittelen. `split(":")` fungerer fint til dette, men husk at alt etter det første kolonet er det du er interessert i.
2. **Datoformatet:** Som nevnt tidligere er `11.9.2001` ikke et format Java forstår automatisk. Du må bruke en `DateTimeFormatter` som matcher `d.M.yyyy`.

Viktig obs: Hvis du bruker `readLine()` flere ganger inne i en `while`-løkke, må du være helt sikker på at filen alltid har riktig antall linjer. Hvis en beskrivelse mangler, vil `readLine()` begynne å lese tittelen på neste serie som om det var beskrivelsen!

Hvilken av disse metodene synes du virker mest logisk for din oppgave? (Å lese linje for linje, eller å prøve å dele opp hele filen i blokker?)

hva gjør `.trim()`

Programmering 2 Chat-Assistent - Egendefinert Gem

ⓘ Dette er bare ment som informasjon. Kontakt lege hvis du trenger medisinske råd eller diagnoser.

Metoden `.trim()` er en veldig praktisk innebygd funksjon i Javas `String`-klasse. Den brukes til å fjerne "whitespace" (mellomrom, tabulatorer, linjeskift) fra både starten og slutten av en tekststreng.

Den rører *ikke* mellomrom som står midt inne i selve teksten.

Hvordan det fungerer:

Her er et generelt eksempel for å illustrere effekten:

Java

```
String dirtyString = "  2024-12-24  ";
String cleanString = dirtyString.trim();

// Resultatet blir "2024-12-24" (uten mellomrommene på sidene)
```

Hvorfor er dette viktig for din oppgave?

I filen din har du linjer som:

```
Release date: 11.9.2001
```

Når du bruker `line.split(":")`, vil den andre delen (`values[1]`) inneholde alt som står etter kolonet. Siden det er et mellomrom rett etter kolonet i fila di, blir verdien `" 11.9.2001"`.

Når du sender dette til `LocalDate.parse()`, vil Java kaste en feil fordi den ser `" "` (mellomrommet) som et ugyldig tegn i en dato. Ved å bruke `.trim()` fjerner du dette usynlige problemet før du prøver å konvertere teksten til et dato-objekt.

Oppsummert:

- **Fjerner:** Mellomrom, `\t` (tab), `\n` (linjeskift) i begge ender.
- **Beholder:** Alt som er mellom de ytterste tegnene (f.eks. mellomrommet i "Breaking Bad" forblir urørt).
- **Returverdi:** Den returnerer en *ny* streng (siden strenger i Java er "immutable" og ikke kan endres direkte).

Vil du se hvordan du kan legge til `.trim()` i koden din for å unngå

`DateTimeParseException`, eller er du klar for å se på hvordan du håndterer datoformatet

`11.9.2001` ?

vis meg hvordan values ser ut fullstendig

Programmering 2 Chat-Assistent - Egendefinert Gem

For å visualisere hvordan `values`-arrayet ser ut, må vi se på hva som skjer nøyaktig når

`line.split(":")` kjøres på de spesifikke linjene i filen din.

Siden filen din har formatet `Nøkkel: Verdi`, vil `split(":")` dele linjen i to deler: alt før kolonet og alt etter kolonet.

Her er en oversikt over hvordan `values` ser ut for de tre første linjene i filen din:

Linje 1: `Title: wawa`

Når denne linjen splittes, får du et array med to elementer:

- `values[0] : "Title"`
- `values[1] : " wawa"` (Legg merke til mellomrommet foran "wawa")

Linje 2: `Release date: 11.9.2001`

Her skjer det samme. Java bryr seg ikke om at det er to ord i "Release date", den ser kun etter kolonet:

- `values[0] : "Release date"`
- `values[1] : " 11.9.2001"` (Igjen, med et mellomrom foran)

Linje 3: `Description: ada kadabra sim sala binz`

Selv om det er mange ord i beskrivelsen, stopper `split` kun ved kolonet:

- `values[0] : "Description"`
- `values[1] : " ada kadabra sim sala binz"`

Oppsummert i en tabell:

Linje i fila	<code>values[0]</code> (Nøkkel)	<code>values[1]</code> (Verdi)
<code>Title: wawa</code>	<code>"Title"</code>	<code>" wawa"</code>
<code>Release date: 11.9.2001</code>	<code>"Release date"</code>	<code>" 11.9.2001"</code>
<code>Description: ada kadabra...</code>	<code>"Description"</code>	<code>" ada kadabra..."</code>

 Eksporter til Regneark 

Hvorfor koden din feilet

I din opprinnelige kode gjorde du dette:

```
LocalDate releaseDate = LocalDate.parse(values[1]);
```

Når løkken din leste den første linjen (`Title: wawa`), inneholdt `values[1]` teksten `" wawa"`.

Siden dette ikke er en dato, krasjet programmet med en gang.

For at koden skal fungere med dette filformatet, må du altså:

- Sørge for at du henter `values[1]` fra riktig linje (den som faktisk inneholder datoen).
- Bruke `.trim()` for å fjerne mellomrommet i `" 11.9.2001"`.
- Bruke en `DateTimeFormatter` fordi datoen bruker punktum (`.`) i stedet for bindestrek (`-`).

Vil du at jeg skal vise deg hvordan du kan sette opp en `DateTimeFormatter` som matcher `11.9.2001` ?


```
@Override public ArrayList<TVSeries> getAllTVSeries() {  
    ArrayList<TVSeries> fetchedTVS = new ArrayList<>();  
    try (BufferedReader bufferedReader = new  
        BufferedReader(new FileReader(file))) {  
        if (file.canRead()) {  
            String line;  
            while ((line = bufferedReader.readLine()) != null) {  
                String[] values = line.split(".");  
                String title = values[1];  
                LocalDate releaseDate = LocalDate.parse(values[1]);  
                String description = values[1];  
                TVSeries newTVS = new TVSeries(title, description,  
                    releaseDate);  
                fetchedTVS.add(newTVS);  
            }  
        }  
    } catch (FileNotFoundException exception) {  
        System.err.println("Could not find file with name " +  
            file.getName());  
    } catch (IOException e) {  
        System.err.println(e.getMessage());  
    }  
    return new ArrayList<>(fetchedTVS);  
}
```

hvordan vil values se ut nå

Programmering 2 Chat-Assistent · Egendefinert Gern

Siden du nå henter `values[1]` for både `title`, `releaseDate` og `description` inne i den samme `while`-løkken, vil verdiene endre seg for hver eneste linje `BufferedReader` leser.

Her er nøyaktig hva som skjer i `values`-arrayet ditt steg for steg når koden kjører på fila di:

Steg 1: Første runde i løkken (Linje 1 i fila)

Linje lest: `Title: wawa`

- `values[0]` = `"Title"`
- `values[1]` = `" wawa"`
- Resultat:** Koden prøver å sette både `title`, `releaseDate` og `description` til `" wawa"`. Programmet krasjer her fordi `LocalDate.parse(" wawa")` er umulig.

Steg 2: Hvis vi later som steg 1 fungerte og går til neste linje

Linje lest: `Release date: 11.9.2001`

- `values[0]` = `"Release date"`
- `values[1]` = `" 11.9.2001"`
- Resultat:** Nå ville alle tre variablene blitt `" 11.9.2001"`. Men koden din ville fortsatt krasjet fordi den mangler `.trim()` og en `DateFormat`.

Oppsummeret tabell over `values[1]` i din nåværende kode:

Når løkken leser...	<code>title</code> blir:	<code>releaseDate</code> prøver å bli:	<code>description</code> blir:
Linje 1 (<code>Title: wawa</code>)	<code>" wawa"</code>	KRA SJ (Tatt 'wawa' could not be parsed)	<code>" wawa"</code>
Linje 2 (<code>Release date: ...</code>)	<code>" 11.9.2001"</code>	KRA SJ (Mangler trim-format)	<code>" 11.9.2001"</code>
Linje 3 (<code>Description: ...</code>)	<code>" ada kadabra..."</code>	KRA SJ (Ikke en dato)	<code>" ada kadabra..."</code>

Eksporter til Ragnark

Hva er problemet?

Problemet er at `values[1]` kun inneholder én av delene du trenger (enten tittel, dato eller beskrivelse), men koden din prøver å fylle ut et helt `TVSeries`-objekt hver gang den leser en linje.

For å fikse dette må du "samle opp" informasjonen. Du kan ikke kalle `new TVSeries(...)` før du har vært gjennom alle tre linjene.

Ser du hvordan du kan endre logikken slik at du henter ut én ting av gangen, og venter med å lage `new TVSeries` til du har alt du trenger?

Tips: Siden du vet at fila din alltid følger mønsteret "Tittel, Dato, Beskrivelse", kan du bruke `bufferedReader.readLine()` tre ganger inne i løkken for å fylle variablene før du lager objektet. Skal vi se på hvordan man kan strukturere det?


```
String line;
while ((line = bufferedReader.readLine()) != null) {
    String[] values = line.split(":");
    String title = values[1];
    bufferedReader.readLine();
    LocalDate releaseDate = LocalDate.parse(values[1]);
    bufferedReader.readLine();
    String description = values[1];
    TVSeries newTVS = new TVSeries(title, description,
    releaseDate);
    fetchedTVS.add(newTVS);
}
```

Programmering 2 Chat-Assistent - Egendefinert Gem

Her ser vi en klassisk logisk utfordring når man jobber med fil-lesing. Selv om du nå har lagt til `bufferedReader.readLine()` inne i løkken for å "gå videre", så er det et par ting som gjør at verdiene dine ikke blir det du forventer.

Her er en oversikt over hvordan `values` og variablene dine ser ut i denne koden:

Hva skjer i koden din nå?

1. **Første** `line = bufferedReader.readLine()`: Java leser "Title: wawa".
2. `values = line.split(":")`: Dette arrayet inneholder nå `["Title", " wawa"]`.
3. `String title = values[1]`: `title` blir " wawa".
4. `bufferedReader.readLine()`: Programmet leser linjen med "Release date", men du lagrer den ikke i en variabel. Den blir bare "lest og glemt".
5. `LocalDate releaseDate = LocalDate.parse(values[1])`: Her er problemet. Du bruker fortsatt `values[1]` fra den første linjen (" wawa"). Du har ikke laget et nytt `values`-array for den nye linjen.
6. `bufferedReader.readLine()`: Programmet leser linjen med "Description", men også denne blir glemt.
7. `String description = values[1]`: Igjen bruker du `values[1]` fra den første linjen (" wawa").

Slik vil variablene se ut for hvert objekt:

Variabel	Verdi i din kode	Hvorfor?
<code>title</code>	" wawa"	Hentet fra <code>values[1]</code> på første linje.
<code>releaseDate</code>	" wawa"	KRA SJU Du bruker fortsatt <code>values[1]</code> fra første linje.
<code>description</code>	" wawa"	Du bruker fortsatt <code>values[1]</code> fra første linje.

 Eksporter til Ragneark



Hvordan leser du dette?

For hver gang du kaller `bufferedReader.readLine()`, må du oppdatere `line`-variabelen og kjøre `split(":")` på nytt for å få de nye verdiene inn i `values`-arrayet.

Et lite tankeeksperiment:

Hvis du skal flytte tre bokser, kan du ikke bare si "flytt boks" tre ganger og forvente at du holder i den tredje boksen hvis du aldri slapp den første. Du må aktivt gripe tak i (bildele til variabel) den nye linjen hver gang.

Prøv å endre logikken slik at du lagrer resultatet av hver `readLine()` i en variabel, og splitter den før du henter ut verdien.

Et spørsmål til deg:

Hvordan tror du koden bør se ut hvis du vil at `releaseDate` skal hente data fra den neste linjen i stedet for den samme som `title`?

```

while ((line = bufferedReader.readLine()) != null) {
    String title = line.split(":")[1];
    String dateLine = bufferedReader.readLine();
    String dateString = dateLine.split(":")[1];
    String descLine = bufferedReader.readLine();
    String description = descLine.split(":")[1];
    LocalDate releaseDate = LocalDate.parse(dateString);
    TVSeries newTVS = new TVSeries(title, description,
    releaseDate);
    fetchedTVS.add(newTVS);
}

```

Exception in thread "main"

java.time.format.DateTimeParseException: Text

'11.9.2001' could not be parsed at index 0

at java.base/

java.time.format.DateTimeFormatter.parseResolved0(DateTimeFormatter.java:2108)

at java.base/

java.time.format.DateTimeFormatter.parse(DateTimeFormatter.java:2010)

at java.base/

java.time.LocalDate.parse(LocalDate.java:437)

at java.base/

java.time.LocalDate.parse(LocalDate.java:422)

at

oblig4.modules.TVSeriesCSVRepository.getAllTVSeries(TVSeriesCSVRepository.java:39)

at oblig4.CSVMain.main(CSVMain.java:18)

hva nå

Nå er du veldig nær! Logikken for å lese de tre forskjellige linjene ser mye bedre ut.

Feilmeldingen du får nå: `Text '11.9.2001' could not be parsed at index 0`, skyldes to spesifikke ting som vi har vært inne på:

1. **Mellomrommet:** Selv om det ikke synes så godt i feilmeldingen, inneholder `dateString` sannsynligvis fortsatt det mellomrommet som står etter kolonet i fila (`" 11.9.2001"`).
2. **Datoformatet:** Standard-metoden `LocalDate.parse()` forventer formatet `YYYY-MM-DD` (feks. `2001-09-11`). Siden din dato bruker punktum og har rekkefølgen `dag.måned.år`, skjønner ikke Java automatisk hvordan den skal lese det.

Slik løser du det

Du må bruke en `DateTimeFormatter` for å fortelle Java nøyaktig hvordan datoen i fila di er bygget opp.

Her er de to stegene du trenger:

1. Definer formatet

Før løkken din (eller som en konstant i klassen), lager du en formatter som matcher `11.9.2001`.

```
Java
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("d.M.yyyy");
```

2. Bruk `.trim()` og formatteren

Når du skal parse datoen, fjerner du mellomrommet og sender med formatteren som et argument:

```
Java
// .trim() fjerner mellomrommet foran 11.9.2001
LocalDate releaseDate = LocalDate.parse(dateString.trim(), formatter);
```

Et lite varsel om den tomme linjen

I filoversikten din så vi at det var en tom linje mellom hver TV-serie:

```
PlainText
Description: ...
(tom linje)
Title: ...
```

Hvis koden din leser tre linjer i hver runde av løkken, vil den neste `while`-sjekken lese den tomme linjen inn i `line`. Da vil `line.split(":")` krasje fordi den tomme linjen ikke inneholder et kolon.

Hvordan kan du håndtere dette?

Du kan legge til en sjekk øverst i løkken for å hoppe over linjer som ikke inneholder tekst:

```
Java
while ((line = bufferedReader.readLine()) != null) {
    if (line.isBlank()) {
        continue; // Hopper til neste runde i while-løkke
    }
    // Resten av koden din her...
}
```

Hva skjer hvis du legger til `.trim()` og `DateTimeFormatter` nå – forevinner feilmeldingen?



men den har ikke mellomrom:

Title:wawa

Release date:11.9.2001

Description:ada kadabra sim sala binz

Skjul teksten

Title:nya

Release date:22.2.2222

Description:neko

Programmering 2 Chat-Assistant • Egendefinert Gem

Ah, skjønner! Hvis fila er formatert uten mellomrom rett etter kolonet (som `Release date:11.9.2001`), så er det kun selve datoformatet som er utfordringen.

Når du bruker `LocalDate.parse("11.9.2001")`, forventer Java at strengen ser ut som `2001-09-11`. Siden datoen din bruker punktum og har en annen rekkefølge, må du fortelle Java nøyaktlig hvordan den skal lese de ulike delene (dag, måned, år).

Løsningen: DateTimeFormatter

Du må bruke en `DateTimeFormatter` for å "oversette" tekststrengen til et `LocalDate`-objekt. Her er de tekniske symbolene Java bruker:

- `d` = Dag (uten ledende null, f.eks. 1 eller 11)
- `M` = Måned (uten ledende null, f.eks. 9 eller 12)
- `yyyy` = År (fire siffer)

Slik implementerer du det i koden din:

Java



```
// 1. Lag formatteren (legg denne gjerne utenfor låkken)
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("d.M.yyyy");

// 2. Bruk formatteren i parse-metoden
LocalDate releaseDate = LocalDate.parse(dataString, formatter);
```

Hva skjer med de tomme linjene?

Siden du bruker `readLine()` tre ganger inne i `while`-løkken, vil programmet ditt lese:

1. Linje med tittel
2. Linje med dato
3. Linje med beskrivelse

Men hva skjer når løkken starter på nytt?

I eksempelet ditt er det et linjeskift (en tom linje) mellom hver serie.

- Eller at den har lest "Description", vil `while ((line = bufferedReader.readLine()) != null)` lese den tomme linjen.
- Da vil `line.split(":")[1]` kaste en ny feil (`ArrayIndexOutOfBoundsException`), fordi en tom linje ikke har et kolon eller en del nummer lo.

Mitt råd for neste steg:

Legg til en sjekk øverst i `while`-løkken som hopper over linjer som er tomme. Det kan gjøres slik:

Java



```
while ((line = bufferedReader.readLine()) != null) {
    if (line.isBlank()) {
        continue; // Går rett til neste readLine()
    }
    // Her legger du koden som splitter tittel, dato og beskrivelse
}
```

Vil du prøve å legge til `DateTimeFormatter` og se om `DateTimeParseException` forevinner?

```

la til :
private static final DateTimeFormatter correctFormat =
    DateTimeFormatter.ofPattern("dd.MM.yyyy");

String title = line.split("-")[1];
String dateLine = bufferedReader.readLine();
String dateString = dateLine.split("-")[1];
String descLine = bufferedReader.readLine();
String description = descLine.split("-")[1];
LocalDate releaseDate =
    LocalDate.parse(dateString, correctFormat);
TVSeries newTVS = new TVSeries(title, description,
    releaseDate);
fetchedTVS.add(newTVS);

får dette:Exception in thread "main"
java.time.format.DateTimeParseException: Text
'11.9.2001' could not be parsed at index 3
    at java.base/
java.time.format.DateTimeFormatter.parseResolved0(Dat
eTimeFormatter.java:2108)
    at java.base/
java.time.format.DateTimeFormatter.parse(DateTimeForm
atter.java:2010)
    at java.base/
java.time.LocalDate.parse(LocalDate.java:437)
    at
oblig4.modules.TVSeriesCSVRepository.getWithTVSeries(T
VSeriesCSVRepository.java:41)
    at oblig4.CSVMain.main(CSVMain.java:18)

```

Programmering 2 Chat-Assistent • Egendefinert Gern

Nå er du ekstremt nært Feilmeldingen `index 3` er faktisk et veldig godt tegn – det betyr at Java klarer å lese de første tegnene, men stoppet opp når den kom til punktum etter måneden.

Hvorfor feiler den på Index 3?

Du har definert mønsteret som `"dd.MM.yyyy"`.

- `dd` betyr at Java forventer **to** siffer for dag (f.eks. `01`, `11`).
- `MM` betyr at Java forventer **to** siffer for måned (f.eks. `09`, `12`).

I datoen din `11.9.2001` er tallet for måneden (`9`) bare **ett** siffer. Siden mønsteret ditt ber om to siffer (`MM`), forventer Java at tegn nummer 4 skal være et tall, men det finner et punktum i stedet. Det er derfor den kaster feil på index 3.

Løsningen

For å gjøre formatteren din mer fleksibel slik at den takler både ett og to siffer (f.eks. både `9` og `12`), bør du bruke enkelt-bokstaver i mønsteret:

- Bruk `d` i stedet for `dd`.
- Bruk `M` i stedet for `MM`.

Prøv å endre definisjonen din til dette:

```

Java
private static final DateTimeFormatter correctFormat = DateTimeFormatter.ofPattern("d.M.

```

En liten sjekk til:

Selv om du sa at det ikke er mellomrom i fila, er det ofte slik at tekstfiler kan ha usynlige mellomrom eller at man ser feil. Siden feilmeldingen nå sier `index 3`, tyder det på at den leste `11.` helt fint, men stoppet på måneden.

Tips: Hvis du endrer til `"d.M.yyyy"` og den fortsatt gir feil (kanskje på `index 0` igjen), så betyr det at det er et mellomrom der likevel, og da må du legge på `.trim()` på `dateString`.

Hva skjer hvis du endrer `dd.MM` til `d.M`? Går den gjennom nå?

Neste steg:

Når du har fått parsing til å fungere, sjekk om lekken din håndterer den tomme linjen mellom seriene. Hvis programmet krasjer rett etter den første serien er lest, er det sannsynligvis fordi `line.split("-")` kjører på en tom linje. Da vet du at `if (line.isBlank()) continue;` må på plass!

Exception in thread "main"

java.lang.ArrayIndexOutOfBoundsException: Index 1 out of bounds for length 1
at oblig4.modules.TVSeriesCSVRepository.getWithTVSeries(TVSeriesCSVRepository.java:36)
at oblig4.CSVMain.main(CSVMain.java:18)

Programmering 2 Chat-Assistent • Egendefinert Gern

Der kom den feilmeldingen vi snakket om!

`ArrayIndexOutOfBoundsException: Index 1 out of bounds for length 1` betyr at du prøver å hente ut `values[1]`, men arrayet har bare ett element. Dette skjer fordi `split(":")` kjøres på en linje som ikke inneholder kolon.

I fila di er dette den tomme linjen mellom TV-seriene. Når `bufferedReader.readLine()` leser den tomme linjen, finner ikke `split(":")` noe kolon, og resultatet blir et array som kun inneholder den tomme strengen i posisjon `[0]`.

Hvordan fikse det?

Du må legge inn en sjekk i starten av `while`-løkka som hopper over linjer som er tomme eller som ikke inneholder data.

Slik kan du strukturere logikken:

Java

```
while ((line = bufferedReader.readLine()) != null) {  
    // 1. Sjekk om linjen er tom eller bare inneholder mellomrom  
    if (line.isBlank()) {  
        continue; // Hopper rett til neste runde i while-løkka og leser neste linje  
    }  
  
    // 2. Nå vet vi at linjen har innhold, så vi kan splitte tittel  
    String title = line.split(":")[1].trim();  
  
    // 3. Les de neste linjene (dato og Beskrivelse)  
    String dateline = bufferedReader.readLine();  
    // Her bør du kanskje også sjekke om dateline er null før du splitter  
    String dateString = dateline.split(":")[1].trim();  
  
    String descLine = bufferedReader.readLine();  
    String description = descLine.split(":")[1].trim();  
  
    // 4. Parse dato og lagre objektet  
    LocalDate releaseDate = LocalDate.parse(dateString, correctFormat);  
    TVSeries newTVS = new TVSeries(title, description, releaseDate);  
    fetchedTVS.add(newTVS);  
}
```

Hvorfor bruke `.trim()` her også?

Selv om du sa at det ikke er mellomrom i fila, er det en "best practice" når man leser tekstfiler. Det gjør koden din robust. Hvis noen ved et uhell trykker på mellomromstasten i tekstfila, vil ikke programmet ditt krasje.

Oppsummering av feilen:

- `while`-løkka di leser den tomme linjen.
- `line.split(":")` klarer ikke å dele den opp.
- `values[1]` eksisterer ikke for den tomme linjen.
- **Løsning:** Bruk `if (line.isBlank()) continue;`.

Fungerer det bedre når du hopper over de tomme linjene?

CSVMain

JAVA

TVSeriesCS...Repository

JAVA

det fungerer men output er tom

Programmering 2 Chat-Assistent - Egendefinert Gem

Analyse ▾

Det er kjempebra at du har fått bukt med feilmeldingene! At programmet nå kjører uten å krasje, betyr at logikken for lesing og parsing av datoer endelig sitter.

Grunnen til at outputen din er tom, skyldes sannsynligvis en liten detalj i `main`-metoden din.

Hvorfor ser du ingenting?

I `CSVMain.java` gjør du følgende:

1. Du oppretter to serier.
2. Du skriver dem til filen ved hjelp av `addListOfTVSeries`.
3. Du kaller `sendFile.getAllTVSeries()`.

Metoden `getAllTVSeries()` returnerer en `ArrayList<TVSeries>`, men den skriver ingenting til konsollen (terminalen) av seg selv. Når metoden er ferdig kjørt, blir listen den returnerte bare "liggende i løse lufta" fordi den ikke lagres i en variabel eller sendes til en `System.out.println()`.

Slik kan du sjekke resultatet

For å faktisk se at koden din har hentet ut dataene, må du lagre resultatet og printe det ut. Du kan endre slutten av `main` til noe som dette:

Java



```
// Lagre listen som blir returnert fra fil-lesingen
ArrayList<TVSeries> result = sendFile.getAllTVSeries();

// Gå gjennom listen og print ut titlene for å bekrefte
System.out.println("Antall serier lest fra fil: " + result.size());

for (TVSeries tvs : result) {
    System.out.println("Lest fra fil: " + tvs.getTitle() + " (" + tvs.getReleaseDate() +
}
```

Et lite tips for robust koding

Når du bruker `bufferedReader.readLine()` inne i en løkke, som du gjør nå, er det alltid en liten risiko for at man når slutten av filen ("EOF") midt i et objekt.

Hvis filen din for eksempel har en tittel på siste linje, men mangler dato eller beskrivelse, vil `dateLine` eller `descLine` bli `null`. Da vil `.split(":")` kaste en ny feilmelding.

Refleksjonsspørsmål:

Hvordan kan du sikre deg at `dateLine` og `descLine` faktisk inneholder tekst før du prøver å splitte dem?

Vil du at jeg skal hjelpe deg med å sette opp en slik sjekk, eller vil du først teste om

`System.out.println` bekrefter at dataene faktisk er der?

Autor

✖ JAVA

Director

✖ JAVA

Person

✖ JAVA

Produkt

✖ JAVA

disse var oppgavene som jeg svarte på:

Oppgave 2.3 - TVSeriesCSVRepository

Opprett en ny chat-logg.

Denne oppgaven består av flere deloppgaver. Velg selv hvordan du...

Programmering 2 Chat-Assistent • Egendefinert Gem

Analyse ▾

Takk for at du deler både koden og oppgaveteksten! Det er veldig nyttig å se helheten. Du har jobbet deg gjennom noen klassiske utfordringer med fil-håndtering og parsing, og resultatet ser nå ut til å fungere teknisk sett.

Her er en konstruktiv tilbakemelding på arbeidet ditt opp mot oppgavetekstens krav:

1. Tilbakemelding på oppgaveløsningen

- Implementasjon av interface:** Du har korrekt implementert `TVSeriesRepository` og overstyrt alle nødvendige metoder.
- Filstruktur (Punkt b):** Oppgaven ber om at hvert objekt lagres på hver sin linje med et fast skille tegn (typisk CSV-format). Din løsning lagrer én serie over fire linjer (Title, Release date, Description, og en tom linje).
 - Kommentar:* Selv om din løsning teknisk sett fungerer for din egen `getAllTVSeries`, bryter den litt med det tradisjonelle CSV-prinsippet (Comma Separated Values) hvor én linje tilsvarer én "record". Hvis du vil følge standarden tettere, kunne du lagret alt på én linje, f.eks: `Tittel;Beskrivelse;Dag;Måned;År`.
- Dato-håndtering:** Du fulgte hintet om å lagre datoen som individuelle tall (dag, måned, år) i `addListOfTVSeries`. Dette er en robust måte å gjøre det på.
- Søketfunksjon (Punkt d):** `getTVSeriesByTitle` fungerer fint ved at den gjenbruker `getAllTVSeries`. Dette er god gjenbruk av kode!

2. Kodekvalitet og tips (Objektorienterte prinsipper)

- Robusthet i `getAllTVSeries`:** * Du bruker `line.split(";")[1]`. Hvis en linje mot formodning skulle mangle et kolon, vil dette kaste en `ArrayIndexOutOfBoundsException`.
 - Tips:** Du kan vurdere å sjekke om `values.length > 1` før du akseesserer indeks 1.
- Bruk av `LocalDate` og `DateTimeFormatter`:**
 - Veldig bra bruk av `DateTimeFormatter` for å håndlere ditt eget format.
 - I `addListOfTVSeries` bygger du dato-strengen manuelt med punktum. Siden du allerede har en `correctFormat` i klassen, kunne du forenklet dette ved å bruke `TVS.getReleaseDate().format(correctFormat)`.
- Innkapsling:** * I `Production.java` og `TVSeries.java` har du god bruk av private instansvariabler og offentlige get/set-metoder.
 - Metoden `getEpisodes()` returnerer en kopi av listen (`new ArrayList<>(episodes)`), noe som er veldig bra for å hindre at kode utenfor klassen endrer på den interne listen utenom `addEpisode`-metoden.

3. Småting å merke seg

- Filnavn:** Du har kalt filen din `"kys.csv"` i `CSVMain`. I et profesjonelt eller akademisk miljø er det lurt å bruke nøytrale filnavn som f.eks. `"tvseries.csv"` (slik oppgaven foreslo) for å opprettholde en profesjonell tone.
- To String-metoder:** Du har laget gode `toString`-metoder som gir oversiktlig output. Dette gjør debugging (som vi så i stad) mye enklere!

Konklusjon

Du har løst en kompleks oppgave som involverer både arv (`Production` → `TVSeries`), fil-IO, og parsing av data. Logikken din for å lese over flere linjer er nå solid etter at du la inn sjekken for tomme linjer.

Er det noe spesielt du føler du lærte av denne feilsøkningsprosessen, eller er det andre deler av koden du vil gå dypere inn i?



"Oppgave 1 - Ord og begreper"

Det er ikke nødvendig å levere noe eget dokument for teorien i denne obligen, men du BØR knytte alle disse konseptene inn i forklaringene dine rundt koden du produserer i programmering-oppgavene. Alltså som en del av chatloggene dine. Dobbeltsjekk at du har vært innom alle konseptene før du leverer.

Skipet blokken

Maven og Byggeverktøy
Abstrakt (klasse og metode)
Interface
Exception
Persistent lagring
CSV
JSON*

for forrige oppgave så er csv, exception og (tror jeg) persistent lagring relevant. csv står for comma separated values og er en filtype som lagrer data i nøkkel-verdi format med en skille tegn (vanligvis komma) imellom. exception er feilmelding som skjer vis en feil i koden oppstår. denne unngår programmet fra å krasje og gi beskjed hvor og hva feilen er. persistent lagring vet jeg ikke helt, men er det ikke bare lagring av data ofte?

Programmering 2 Chat-Assistent • Egendefinert Gern

Det er helt riktig observert at disse konseptene har vært sentrale i arbeidet ditt så langt! Du har allerede brukt dem i praksis, noe som er den beste måten å lære på.

Her er en liten oppklaring og utfylling av begrepene dine, slik at de sitter helt støpt til innleveringen:

CSV (Comma Separated Values)

Du har rett i at det brukes som et "nøkkel-verdi"-format i din oppgave, men teknisk sett er CSV ment for **tabulære data** (rader og kolonner).

- **Din bruk:** Du brukte `;` som skille tegn. Selv om det heter "Comma Separated", kan man bruke semikolon, tab eller kolon, så lenge programmet vet hva det skal lete etter.
- **Struktur:** Vanligvis representerer hver linje ett objekt (f.eks. én TV-serie).

Exception (Unntak)

En **Exception** er en hendelse som avbryter den normale flyten i programmet.

- **Håndtering:** Du brukte `try-catch` blokker. Som du sier, hindrer dette programmet i å krasje ("fatal error") ved at du fanger feilen og gir en kontrollert beskjed til brukeren, for eksempel hvis filen ikke finnes (`FileNotFoundException`).

Persistent lagring

Du er inne på det! **Persistent lagring** betyr at dataene **overlever** selv om programmet avsluttes eller strømmen går.

- **I din oppgave:** Når du skriver til en `.csv`-fil på harddisken, lager du persistent lagring.
- **Motsetning:** Data som bare ligger i en `ArrayList` i minnet (RAM) er **flyktig** (volatile) og forsvinner med en gang programmet lukkes.

De andre konseptene i koden din

For at du skal få krysset av all på listen til oppgave 1, er det verdt å merke seg at du faktisk har brukt flere av de andre konseptene også:

- **Abstrakt klasse:** Du har brukt `public abstract class Production` og `public abstract class Person`. Dette betyr at man ikke kan lage et objekt av typen "Production" direkte, man må lage en spesifikk `TVSeries`.
- **Interface:** Klassen din `TVSeriesCSVRepository` implementerer `TVSeriesRepository`. Dette er en kontrakt som sier hvilke metoder klassen **må** inneholde.
- **Maven:** Dette er verktøyet som styrer prosjektstrukturen din og hjelper deg med å bygge/kjøre koden (hvis prosjektet ditt er satt opp med en `pom.xml`-fil).

En liten rettelse: Du skrev at CSV lagrer data i nøkkel-verdi format. I din spesifikke fil gjorde du det slik (`{ Title:Wawa }`), men i standard CSV lagrer man ofte bare verdiene (`Wawa,11.09.2001,beskriverWawa`) og lar rekkefølgen eller en overskriftsrad fortelle hva som er hva.

Vil du at vi skal se på hvordan du kan bruke JSON som et alternativ til CSV for å lagre TV-seriene dine, slik at du får dekket det siste punktet på listen også?



Programmering 2 Chat-Assistent • Egendefinert Gem

Hei! Det er et veldig sentralt spørsmål i programmering, spesielt når man lager applikasjoner som skal brukes over tid.

Persistent lagring (eller *persistens*) refererer til mekanismer som sørger for at data overlever selv etter at programmet avsluttes eller strømmen skrus av.

Når du kjører et Java-program, lagres variabler og objekter i maskinens arbeidsminne (**RAM**). Dette minnet er *flyktig*, som betyr at alt slettes så fort programmet slutter å kjøre. For at informasjonen skal være tilgjengelig neste gang du starter programmet, må den skrives til et ikke-flyktig lagringsmedium, som en harddisk eller en SSD.

Vanlige metoder for persistent lagring i Java:

- **Tekstfiler (.txt):** Enkel lagring av rådata eller konfigurasjon.
- **Serialisering:** En innebygd Java-mekanisme for å lagre hele objekttilstander direkte til en fil.
- **Strukturerte dataformater (JSON/XML):** Brukes ofte for å lagre data på en måte som er lett å lese for både mennesker og andre systemer.
- **Databaser (SQL/NoSQL):** Brukes for større mengder data som krever avansert søking og organisering.

Hvordan henger dette sammen med objektorientering?

I et objektorientert program har vi ofte en utfordring: Vi har objekter i minnet med relasjoner til hverandre, men på disken lagres data ofte som flate filer eller tabeller. Prosessen med å overføre data mellom disse to verdenene kalles ofte *Mapping*.

JSONMain

<> JAVA

TVSeries

<> JAVA

TVSeriesJS...Repository

<> JAVA

TVSeries

<> JAVA

får denne feilmelding:

"could not read TV-series from JSON file: Cannot construct instance of `oblig4.modules.TVSeries` (no Creators, like default constructor, exist): cannot deserialize from Object value (no delegate- or property-...

Programmering 2 Chat-Assistent • Egendefinert Gem

Analysis ▼

Her har du støtt på en veldig vanlig utfordring når man jobber med **JSON-serialisering** i Java, spesifikt med biblioteket Jackson.

Feilmeldingen din peker på to problemer som henger sammen. La oss bryte dem ned:

1. Hovedårsaken: Manglende konstruktør

Feilmeldingen sier: `no Creators, like default constructor, exist`.

Når Jackson skal lese en JSON-fil og lage Java-objekter (deserialisering), gjør den vanligvis følgende:

1. Den oppretter en tom instans av klassen ved å bruke en **standardkonstruktør** (en konstruktør uten parametere).
2. Den bruker *setter*-metoder eller refleksjon for å fylle inn verdiene fra JSON-filen.

I din `TVSeries.java` har du kun definert én konstruktør som tar i mot `title`, `description` og `releaseDate`. Siden du har laget denne, har Java fjernet den automatiske standardkonstruktøren. Jackson vet derfor ikke hvordan den skal "starte" byggingen av et `TVSeries`-objekt fra bunnen av.

2. NullPointerException i Main

Dette er en følgefeil. Fordi lesingen feilet i `TVSeriesJSONRepository`, returnerte metoden `null` (som du har definert i `catch`-blokken).

I `JSONMain.java` prøver du å kjøre en for-each løkke over `readJSON`. Siden `readJSON` er `null`, kaster Java en `NullPointerException`.

Hvordan løse dette?

For å fikse dette må Jackson få en måte å opprette objektene på. Du har to hovedvalg:

Alternativ A: Legg til en tom konstruktør (Enklest)

Legg til en "default constructor" i `TVSeries.java`. Den kan gjerne være `protected` eller `public`.

Java



```
// I TVSeries.java
public TVSeries() {
    // Jackson trenger denne
    super();
}
```

Merk: Siden `TVSeries` arver fra `Production`, må også `Production` ha en tom konstruktør for at dette skal fungere.

Alternativ B: Bruk annotasjoner

Hvis du vil tvinge Jackson til å bruke den konstruktøren du allerede har, kan du bruke `@JsonCreator` og `@JsonProperty` annotasjoner. Dette krever at du importerer Jackson-biblioteket i `TVSeries`-klassen.

Et lite tips om feilhåndtering

I `JSONMain.java` har du denne linjen:

```
assert readJSON != null;
```

Som vi snakket om i forrige svar, kjører `assert` bare hvis du har aktivert `-ea` flagget. Siden programmet ditt krasjet med en `NullPointerException` på linjen etter, betyr det at assertions sannsynligvis ikke er aktivert i din IDE akkurat nå.

For å gjøre koden mer robust mot filfeil, kan det være lurt å bruke en vanlig `if`-test:

Java



```
if (readJSON != null) {
    for (TVSeries fetchedTVS : readJSON) {
        System.out.println(fetchedTVS);
    }
} else {
    System.out.println("Kunne ikke laste data fra fil.");
}
```

Har du mulighet til å legge til en tom konstruktør i både `TVSeries` og forelder-klassen `Production`, eller ønsker du å se hvordan man bruker annotasjoner i stedet?

ⓘ Dette er bare ment som informasjon. Kontakt lege hvis du trenger medisinske råd eller diagnoser.

For å implementere Alternativ B, bruker vi annoteringer fra Jackson-biblioteket. Dette er ofte foretrukket hvis du vil beholde "immutability" (uforanderlighet) eller unngå tomme konstruktører som kan etterlate objekter i en ugyldig tilstand.

Slik gjør du det i `TVSeries.java`:

1. Legg til import-setninger

Du må importere de nødvendige klassene fra Jackson-pakken øverst i filen din:

Java

```
import com.fasterxml.jackson.annotation.JsonCreator;
import com.fasterxml.jackson.annotation.JsonProperty;
```

2. Bruk `@JsonCreator` og `@JsonProperty`

Du må fortelle Jackson nøyaktig hvilke JSON-felt som hører til hvilke parametere i konstruktøren din.

Her er hvordan konstruktøren din bør se ut:

Java

```
@JsonCreator
public TVSeries(
    @JsonProperty("title") String title,
    @JsonProperty("description") String description,
    @JsonProperty("releaseDate") LocalDate releaseDate
) {
    super(title, description, releaseDate);
}
```

Hvorfor er dette nødvendig?

- **Mapping:** I kompilert Java-kode forsvinner ofte navnene på parametere (de blir bare `arg0`, `arg1`, osv.). `@JsonProperty("title")` forteller Jackson at verdien fra JSON-nøkkelen `"title"` skal sendes inn som det første argumentet.
- **Arv:** Siden `TVSeries` arver fra `Production`, vil Jackson bruke denne konstruktøren og sende verdiene videre opp til `super()`.

Viktige forutsetninger

For at dette skal fungere knirkefritt, må du sjekke to ting:

1. **Getter-metoder:** Jackson trenger vanligvis enten public felter eller public "get"-metoder for å kunne lese dataene fra objektet når det skal skrives til fil. Sørg for at `getTitle()`, `getDescription()` og `getReleaseDate()` finnes (sannsynligvis i `Production`-klassen din).
2. **JavaTimeModule:** Du har allerede gjort dette riktig i `TVSeriesJSONRepository.java` ved å registrere `new JavaTimeModule()`. Dette er helt nødvendig for at Jackson skal forstå `LocalDate`.

Et lite tips om feilen i Main

Husk at selv om du fikser konstruktøren, kan `getAllTVSeries()` fortsatt returnere `null` hvis for eksempel filen ikke finnes eller er tom.

I `JSONMain.java` bør du vurdere å endre:

Java



```
assert readJSON != null;
```

til en faktisk sjekk, for å unngå at programmet krasjer hardt:

Java



```
if (readJSON != null && !readJSON.isEmpty()) {  
    // kjør løkken din her  
}
```

Fungerer deserialiseringen bedre nå som du har lagt til disse annoteringene på konstruktøren?

Production

 JAVA

TVSeries

 JAVA

TVSeriesJS...Repository

 JAVA

TVSeries

 JAVA

oppgaver:



"b)

Som du gjorde i Oppgave 2.3 a) tidligere, opprett en instansvariabel, file av typen File, og en konstruktør som tar imot og setter objektet for denne instansvariabelen...

Programmering 2 Chat-Assistent - Egendefinert Gem

Analyse 

Hei! Dette ser ut som et solid stykke arbeid. Du har svart godt på oppgavelekstens krav og implementert de tekniske komponentene som kreves for å håndtere JSON i Java.

Her er en gjennomgang av løsningen din med tilbakemelding på de ulike delene:

Oppgave b) Konstruktør og filhåndtering

Dette er helt korrekt løst. Ved å sende inn `File`-objektet i konstruktøren følger du prinsippet om Dependency Injection, som gjør klassen din mer fleksibel (du kan enkelt bytte fil uten å endre koden i selve repository-klassen).

Oppgave c) og d) JSON-logikk med ObjectMapper

Du har fått med deg de viktigste detaljene her:

- **JavaTimeModule:** Det er helt nødvendig for å håndtere `LocalDate`, ellers ville koden krasjet ved første dato.
- **Pretty Printer:** Dette er en god vane for lesbarhet under utvikling.
- **Tabell-tilnærming:** Bruken av `TVSeries[].class` i `readValue` er en standard og trygg måte å lese lister fra JSON på.

Et lite tips om kodekvalitet:

I både `addListOfTVSeries` og `getAllTVSeries` oppretter du en ny `ObjectMapper` og registrerer modulen hver gang metoden kalles.

Vurdering: For en innlevering er dette helt greit, men for optimalisering kunne du vurdert å ha `ObjectMapper` som en `private static final` variabel i klassen. Siden `ObjectMapper` er "thread-safe" og ressurskrevende å sette opp, er det vanlig praksis å gjenbruke den. Dette er en valgfri optimalisering.

Oppgave e) getTVSeriesByTitle

Løsningen din med å hente hele listen via `getAllTVSeries()` og deretter iterere gjennom den for å finne riktig tittel er den mest logiske måten å gjøre det på når man jobber med flate JSON-filer.

Viktig tilbakemelding på feilhåndtering (Logiske feil)

Jeg ser en potensiell utfordring i `getTVSeriesByTitle`:

Java 

```
for (TVSeries tvs:getAllTVSeries()){
    if (tvs.getTitle().equalsIgnoreCase(title)) {
        fetchedTVS = tvs;
    }
}
```

Hvis `getAllTVSeries()` returnerer `null` (som skjer i din `catch`-blokk hvis filen ikke kan leses), vil denne linjen kaste en `NullPointerException`.

- **Forbedring:** Du bør sjekke om listen du får fra `getAllTVSeries()` faktisk inneholder data før du prøver å gå gjennom den i en løkke.

Oppsummering av krav i forhold til `TVSeries` og `Production`

Du har nå lagt til annotasjonene `@JsonCreator` og `@JsonProperty` i konstruktøren (fra forrige melding). Siden `TVSeries` arver fra `Production`, må du passe på følgende:

1. **Innkapsling:** Sørg for at alle feltene i `Production` (som `title`, `description`, `releaseDate`) har tilhørende getter-metoder. Jackson bruker disse når den skal skrive objektet *til* fil (serialisering). Jeg ser i koden din at du har disse på plass – bra!
2. **Tom konstruktør:** Du har en tom konstruktør i `Production`. Dette er bra for arv, men pass på at alle klasser i hierarkiet støtter måten Jackson bygger objekter på.

Videre steg

Løsningen din fungerer nå bra for det som er forespurt!

Vil du at vi skal se på hvordan du kan gjøre `getTVSeriesByTitle` mer robust for å unngå krasj hvis filen mangler, eller er du klar for å gå videre til neste del av oppgaven?



```
for optimaliseringen kan private static final ObjectMapper  
mapper = JsonMapper.builder()  
.findAndAddModules()  
.build(); fungere
```

Programmering 2 Chat-Assistent - Egendefinert Gem

Ja, det er en utmerket måte å gjøre det på! Ved å bruke en `private static final` instans av `ObjectMapper` (eller `JsonMapper`), oppnår du flere fordeler:

1. **Ytelse:** `ObjectMapper` er en tungvektler å instansiere. Ved å lage den én gang, sparer du ressurser hver gang du skal lese eller skrive til fil.
2. **Trådsikkerhet:** Jackson er designet slik at `ObjectMapper` er trådsikker etter at den er ferdig konfigurert, noe som gjør den trygg å dele som en statisk variabel.
3. **Ryddighet:** Du slipper å gjenta konfigurasjonen (som `registerModule(new JavaTimeModule())`) inne i hver enkelt metode.

Slik kan det se ut i din klasse:

Siden du bruker `JsonMapper.builder().findAndAddModules().build()`, vil Jackson automatisk finne og registrere `JavaTimeModule` for deg, forutsatt at biblioteket ligger i din `pom.xml`.

Java



```
public class TVSeriesJSONRepository implements TVSeriesRepository {  
    private File file;  
  
    // En felles mapper for hele klassen  
    private static final ObjectMapper mapper = JsonMapper.builder()  
        .findAndAddModules() // Finner automatisk JavaTimeModule  
        .build();  
  
    public TVSeriesJSONRepository(File f) {  
        setFile(f);  
    }  
  
    @Override  
    public void addListofTVSeries(ArrayList<TVSeries> listofTVSeries) {  
        try {  
            // Vi bruker den statiske mapperen direkte  
            mapper.writerWithDefaultPrettyPrinter().writeValue(file, listofTVSeries);  
        } catch (IOException e) {  
            System.err.println("could not write TV-series to JSON file: " + e.getMessage());  
        }  
    }  
  
    @Override  
    public ArrayList<TVSeries> getAllTVSeries() {  
        try {  
            // Vi bruker den statiske mapperen direkte  
            TVSeries[] reading = mapper.readValue(getFile(), TVSeries[].class);  
            return new ArrayList<>(Arrays.asList(reading));  
        } catch (IOException e) {  
            System.err.println("could not read TV-series from JSON file: " + e.getMessage());  
        }  
        return new ArrayList<>(); // Returner tom liste i stedet for null for å unngå kr  
    }  
  
    // ... resten av metodene  
}
```

En liten refleksjon rundt feilhåndtering

I din nåværende kode returnerer `getAllTVSeries()` verdien `null` hvis noe går galt. Som vi så i din `JSONMain`, førte dette til en `NullPointerException`.

Et tips for å gjøre koden enda mer robust: Hvis du endrer `return null;` til `return new ArrayList<>();` i `catch`-blokken, vil resten av programmet ditt (som løkker og søk) fortsatt fungere uten å krasje, selv om filen er tom eller mangler.

Det er et veldig godt spørsmål som treffer kjernen i innkapsling (encapsulation), et av de viktigste prinsippene i objektorientert programmering.

Når det gjelder din `private static final ObjectMapper`, er svaret som hovedregel nei, den bør ikke ha hverken getter eller setter. Her er hvorfor:

1. Hvorfor ikke setter?

Siden variabelen er markert som `final`, kan den uansett ikke endres etter at den er initialisert. En setter-metode vil rett og slett ikke fungere teknisk, og logisk sett ønsker vi at alle metoder i klassen skal bruke den samme, ferdigkonfigurerte mappen.

2. Hvorfor ikke getter?

En getter brukes når andre klasser trenger tilgang til en variabel.

- **Internt verktøy:** `ObjectMapper` er i dette tilfellet et internt verktøy for `TVSeriesJSONRepository` for å utføre oppgavene sine (lese/skrive).
- **Skjuling av implementasjon:** Andre deler av programmet ditt (som `JSONMain`) trenger ikke å vite *hvordan* repository-klassen lagrer data, bare at den gjør det. Ved å holde mappen `private`, skjuler du detaljene om at du bruker Jackson-biblioteket. Dette gjør det lettere å bytte ut biblioteket senere uten å måtte endre kode andre steder.

Oppsummering av felt i `TVSeriesJSONRepository`

Basert på koden din, ser det slik ut:

Felt	Tilgang	Getter/Setter?	Begrunnelse
file	private	Ja (begge)	Klassen må vite hvilken fil den skal jobbe mot, og dette kan endres via konstruktøren eller set-metoden.
mapper	private static final	Nei	Dette er en intern, uforanderlig hjelpevariabel som kun denne klassen skal bruke.

Et lite tips for `getTVSeriesByTitle`

Siden du nå er i gang med å rydde i koden, ser jeg at du bruker `getAllTVSeries()` inne i `getTVSeriesByTitle()`. Dette er bra gjenbruk av kode!

For å unngå potensielle krasj hvis filen er tom, kan du gjøre et lite grep i `getTVSeriesByTitle`:

Java



```
@Override
public TVSeries getTVSeriesByTitle(String title) {
    ArrayList<TVSeries> allSeries = getAllTVSeries();

    // Sjekk om listen faktisk inneholder noe før vi itererer
    if (allSeries != null) {
        for (TVSeries tvs : allSeries) {
            if (tvs.getTitle().equalsIgnoreCase(title)) {
                return tvs; // Returner med en gang vi finner den
            }
        }
    }
    return null; // Returner null hvis ingenting ble funnet
}
```

Er det noe annet i oppgaven du er usikker på, for eksempel hvordan du skal håndtere feilmeldinger hvis filen ikke eksisterer når du prøver å lese fra den?